SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code.* *(BL3)*

Assessment Tool Weight-age: 40%

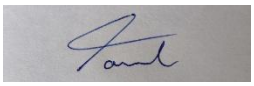
Duration :60 Mins
On Class
QUESTIONS: 5 **Total Marks:50**

Annexure A

SIMULATION TASK

Code of Conduct:

I **Tamil Elakiya S (192524443)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Q. No	Conceptual Understanding (20)	Model Design (25)	Tool Usage(20)	Analysis of Results(20)	Documentation(15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	**/ 25**	**/ 20**	**/ 20**	**/ 15**	**/ 100**

Annexure A

SIMULATION TASK

.**Q1. A compiler receives an intermediate representation of an arithmetic expression containing multiple operators and temporary values.**

Solution :

Simulation Tasks :

- **Simulate instruction selection for the arithmetic operations :**

The intermediate instructions are:

t1 = b + c
t2 = t1 * d
t3 = e - f
t4 = t2 + t3
a = t4

Select appropriate target instructions:

IR Instruction	Target Instruction
t1 = b + c	ADD R1, b, c
t2 = t1 * d	MUL R2, R1, d
t3 = e - f	SUB R3, e, f
t4 = t2 + t3	ADD R2, R2, R3
a = t4	STORE a, R2

- **Allocate registers for operands and temporary values :**
  - `R1 → t1`
  - `R2 → t2 / t4`
  - `R3 → t3`

Register	Contents
R1	`b + c`
R2	`(b+c) × d`
R3	`e - f`

- **Generate the corresponding target instruction sequence :**

```
LOAD R1, b
ADD  R1, c

LOAD R2, R1
MUL  R2, d

LOAD R3, e
SUB  R3, f

ADD  R2, R3
STORE a, R2
```

- **Show the register contents after each instruction :**

Register simulation

Step	Instruction	R1	R2	R3
1	`LOAD R1,b`	10	–	–
2	`ADD R1,c`	30	–	–
3	`MOV R2,R1`	30	30	–
4	`MUL R2,d`	30	60	–
5	`LOAD R3,e`	30	60	50
6	`SUB R3,f`	30	60	35
7	`ADD R2,R3`	30	95	35
8	`STORE a,R2`	30	95	35

- **Evaluate the efficiency of the generated code :**

  Without registers, every temporary value may need to be stored in memory.
  Using registers:

- temporary values remain in CPU registers
- memory access is reduced
- arithmetic operations are faster
- register reuse is possible

Code :

```
b, c, d, e, f = 10, 20, 2, 50, 15

registers = {
   "R1": None,
   "R2": None,
   "R3": None
}
```

```
instructions = [
    "LOAD R1, b",
    "ADD R1, c",
    "MOV R2, R1",
    "MUL R2, d",
    "LOAD R3, e",
    "SUB R3, f",
    "ADD R2, R3",
    "STORE a, R2"
]

registers["R1"] = b
print("1.", instructions[0], registers)

registers["R1"] += c
print("2.", instructions[1], registers)

registers["R2"] = registers["R1"]
print("3.", instructions[2], registers)

registers["R2"] *= d
print("4.", instructions[3], registers)

registers["R3"] = e
print("5.", instructions[4], registers)

registers["R3"] -= f
print("6.", instructions[5], registers)

registers["R2"] += registers["R3"]
print("7.", instructions[6], registers)

a = registers["R2"]
print("8.", instructions[7], registers)

print("\nFinal result: a =", a)
```

Output :

```
1. LOAD R1, b {'R1': 10, 'R2': None, 'R3': None}
2. ADD R1, c {'R1': 30, 'R2': None, 'R3': None}
3. MOV R2, R1 {'R1': 30, 'R2': 30, 'R3': None}
4. MUL R2, d {'R1': 30, 'R2': 60, 'R3': None}
5. LOAD R3, e {'R1': 30, 'R2': 60, 'R3': 50}
6. SUB R3, f {'R1': 30, 'R2': 60, 'R3': 35}
7. ADD R2, R3 {'R1': 30, 'R2': 95, 'R3': 35}
8. STORE a, R2 {'R1': 30, 'R2': 95, 'R3': 35}

Final result: a = 95
```

Q2.A sequence of nested procedure invocations is represented in intermediate form.

Solution:

Simulation Tasks:

• Simulate runtime stack creation:

Consider the nested procedure invocation:

MAIN
↓
P(10)
↓
Q(15)

Procedure P calls procedure Q.

• Construct activation records:

Each activation record contains parameters, return address, control link, local variables and temporaries.

Activation Record for MAIN:

MAIN AR

Parameters : None
Return : OS
Local : None

Activation Record for P:

P AR

Parameter : x = 10
Return : MAIN
Control : MAIN AR
Local : a

Activation Record for Q:

Q AR

Parameter : y = 15
Return : P
Control : P AR
Local : b

• Allocate storage for parameters and local variables:

Assume each parameter and local variable requires 4 bytes.

Procedure	Parameter	Local Variable	Storage
MAIN	None	None	0 bytes
P	x = 10	a	8 bytes
Q	y = 15	b	8 bytes

For the complete activation record, assume return address = 4 bytes and control link = 4 bytes.

Activation Record	Parameter	Return Address	Control Link	Local Variable	Total
P	4 bytes	4 bytes	4 bytes	4 bytes	16 bytes
Q	4 bytes	4 bytes	4 bytes	4 bytes	16 bytes

• Explain the role of runtime storage management during code generation:

Runtime storage management is responsible for:

• Allocating memory for activation records.

• Storing parameters and local variables.

• Maintaining return addresses.

• Maintaining control links between procedures.

Code:

```
stack = []

def display_stack():
    print("\nCurrent Runtime Stack:")
    if not stack:
        print("EMPTY")
    else:
        for record in reversed(stack):
            print(record)

# CALL MAIN
stack.append({
    "Procedure": "MAIN",
    "Parameters": "-",
    "Local Variables": "-"
})
print("CALL MAIN")
display_stack()
```

```
# CALL P(10)
stack.append({
   "Procedure": "P",
   "Parameters": "x = 10",
   "Local Variables": "a"
})
print("\nCALL P(10)")
display_stack()

# CALL Q(15)
stack.append({
   "Procedure": "Q",
   "Parameters": "y = 15",
   "Local Variables": "b"
})
print("\nCALL Q(15)")
display_stack()

# RETURN Q
stack.pop()
print("\nRETURN Q")
display_stack()

# RETURN P
stack.pop()
print("\nRETURN P")
display_stack()

# RETURN MAIN
stack.pop()
print("\nRETURN MAIN")
display_stack()
```

Output:

```
Current Runtime Stack:
{'Procedure': 'MAIN', 'Parameters': '-', 'Local Variables': '-'}

CALL P(10)

Current Runtime Stack:
{'Procedure': 'P', 'Parameters': 'x = 10', 'Local Variables': 'a'}
{'Procedure': 'MAIN', 'Parameters': '-', 'Local Variables': '-'}

CALL Q(15)

Current Runtime Stack:
{'Procedure': 'Q', 'Parameters': 'y = 15', 'Local Variables': 'b'}
{'Procedure': 'P', 'Parameters': 'x = 10', 'Local Variables': 'a'}
{'Procedure': 'MAIN', 'Parameters': '-', 'Local Variables': '-'}

RETURN Q

Current Runtime Stack:
{'Procedure': 'P', 'Parameters': 'x = 10', 'Local Variables': 'a'}
{'Procedure': 'MAIN', 'Parameters': '-', 'Local Variables': '-'}

RETURN P

Current Runtime Stack:
{'Procedure': 'MAIN', 'Parameters': '-', 'Local Variables': '-'}

RETURN MAIN

Current Runtime Stack:
EMPTY
```

Q3. Next-use information for variables is available before code generation begins.

Solution:

Simulation Tasks:

• Construct the next-use table:

Consider the intermediate instructions:
1. a = b + c
2. d = a + e
3. f = d * g
4. h = f + i
The next-use information is:

Statement	Variable	Next Use
1	a	2
1	b	None
1	c	None
2	d	3
2	e	None
3	f	4
3	g	None
4	h	None
4	i	None

• Simulate register allocation based on next-use information:

Assume two registers: **R1 and R2**.

Statement	Operation	R1	R2
1	a = b + c	a	–
2	d = a + e	d	–
3	f = d × g	f	g
4	h = f + i	h	i

Register allocation:

R1 → a → d → f → h

R2 → g / i

• Release registers whenever values become inactive:

Variable	Last Use	Action
b	1	Release
c	1	Release
a	2	Release
e	2	Release
d	3	Release
g	3	Release
f	4	Release
i	4	Release

• Generate the final target instruction sequence:

```
LOAD R1, b
ADD  R1, c
ADD  R1, e
LOAD R2, g
MUL  R1, R2
LOAD R2, i
ADD  R1, R2
STORE h, R1
```

• Explain how next-use information improves register utilization:

Next-use information tells the compiler when a variable will be used again.

If a variable has no future use, its register can be released and reused.

Benefits:

• Better register utilization.

• Reduced unnecessary memory accesses.

• Reduced register spilling.

• Reduced instruction count.

• Improved execution speed.

Therefore, next-use information helps the compiler allocate registers efficiently.

Code:

```
instructions = [
    "a = b + c",
    "d = a + e",
    "f = d * g",
    "h = f + i"
]

next_use = {
    "a": 2,
    "d": 3,
    "f": 4,
    "b": None,
    "c": None,
    "e": None,
    "g": None,
    "i": None
}

registers = {
    "R1": None,
    "R2": None
}

print("NEXT-USE TABLE")
print("----------------")

for variable, use in next_use.items():
    print(variable, "->", use)

print("\nREGISTER ALLOCATION")

registers["R1"] = "a"
print("Statement 1: R1 =", registers["R1"])

registers["R1"] = "d"
print("Statement 2: R1 =", registers["R1"])

registers["R1"] = "f"
registers["R2"] = "g"
print("Statement 3: R1 =", registers["R1"],
      ", R2 =", registers["R2"])

registers["R2"] = "i"
print("Statement 4: R1 =", registers["R1"],
      ", R2 =", registers["R2"])

registers["R1"] = "h"
```

```
print("\nFINAL REGISTER STATE")
print(registers)

print("\nFinal Target Code:")
print("LOAD R1, b")
print("ADD R1, c")
print("ADD R1, e")
print("LOAD R2, g")
print("MUL R1, R2")
print("LOAD R2, i")
print("ADD R1, R2")
print("STORE h, R1")
```

Output:

```
NEXT-USE TABLE
----------------
a -> 2
d -> 3
f -> 4
b -> None
c -> None
e -> None
g -> None
i -> None

REGISTER ALLOCATION
Statement 1: R1 = a
Statement 2: R1 = d
Statement 3: R1 = f , R2 = g
Statement 4: R1 = f , R2 = i

FINAL REGISTER STATE
{'R1': 'h', 'R2': 'i'}

Final Target Code:
LOAD R1, b
ADD R1, c
ADD R1, e
LOAD R2, g
MUL R1, R2
LOAD R2, i
ADD R1, R2
STORE h, R1
```

Q4. An intermediate instruction sequence contains repeated computations inside a loop.

Solution:

Simulation Tasks:

• Identify computations repeated during loop execution:

Consider the intermediate instruction sequence:

```
for (i = 0; i < n; i++)
{
   t1 = a * b;
```

```
    t2 = t1 + i;
    c[i] = t2;
}
```

The computation:

t1 = a * b

is repeated during every loop iteration.

Since a and b do not change inside the loop, a * b is a loop-invariant computation.

For n = 1000, the multiplication is performed **1000 times**.

• Move loop-invariant computations outside the loop:

Before optimization:

```
for (i = 0; i < n; i++)
{
    t1 = a * b;
    t2 = t1 + i;
    c[i] = t2;
}
```

After optimization:

```
t1 = a * b;

for (i = 0; i < n; i++)
{
    t2 = t1 + i;
    c[i] = t2;
}
```

• Generate the optimized loop:

Before optimization:

```
L1:
t1 = a * b
t2 = t1 + i
c[i] = t2
i = i + 1
if i < n goto L1
```

After optimization:

```
t1 = a * b

L1:
t2 = t1 + i
c[i] = t2
i = i + 1
if i < n goto L1
```

• Compare the execution cost before and after optimization:

Assume:

n = 1000

Operation	**Before Optimization**	**After Optimization**
Multiplication	1000	1
Addition	1000	1000
Total Arithmetic Operations	2000	1001
Operations Saved	–	999

Therefore:

Multiplications saved = 1000 - 1

= 999

• Explain why loop optimization improves performance:

Loop optimization improves performance by moving computations that do not change during loop execution outside the loop.

Benefits:

• Reduces repeated calculations.

• Reduces CPU workload.

• Reduces execution time.

• Produces efficient target code.

• Reduces power consumption.

Code:

```
n = 1000

# Before optimization
multiplications_before = 0
additions_before = 0

for i in range(n):
    t1 = 5 * 10
    multiplications_before += 1

    t2 = t1 + i
    additions_before += 1

# After optimization
multiplications_after = 0
additions_after = 0

t1 = 5 * 10
multiplications_after += 1

for i in range(n):
    t2 = t1 + i
    additions_after += 1

print("BEFORE OPTIMIZATION")
print("Multiplications:", multiplications_before)
print("Additions:", additions_before)
print("Total:",
      multiplications_before + additions_before)

print("\nAFTER OPTIMIZATION")
print("Multiplications:", multiplications_after)
print("Additions:", additions_after)
print("Total:",
      multiplications_after + additions_after)

saved = (
    (multiplications_before + additions_before)
    -
    (multiplications_after + additions_after)
)

print("\nOperations saved:", saved)
```

Output:

```
BEFORE OPTIMIZATION
Multiplications: 1000
Additions: 1000
Total: 2000

AFTER OPTIMIZATION
Multiplications: 1
Additions: 1000
Total: 1001

Operations saved: 999
```

Q5. A basic block contains multiple repeated arithmetic expressions.

Solution:

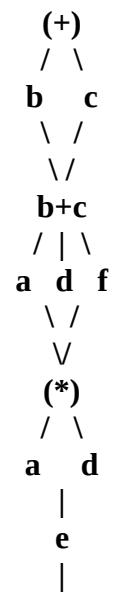
Simulation Tasks:

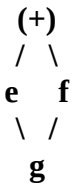
• Construct the Directed Acyclic Graph (DAG):

Consider the basic block:

a = b + c
d = b + c
e = a * d
f = b + c
g = e + f

.





• Identify common subexpressions:

The repeated arithmetic expression is:

b + c

It occurs three times:

a = b + c
d = b + c
f = b + c

Therefore, b + c is identified as a common subexpression.

• Eliminate redundant computations:

Original:

a = b + c
d = b + c
e = a * d
f = b + c
g = e + f

Calculate b + c only once:

t1 = b + c
a = t1
d = t1
e = a * d
f = t1
g = e + f

The expression b + c is now evaluated only once.

• Generate optimized intermediate instructions from the DAG:

a = t1
d = t1
f = t1

the optimized intermediate code can be written as:

t1 = b + c
e = t1 * t1
g = e + t1

• Compare the original and optimized instruction sequences:

Original:

```
a = b + c
d = b + c
e = a * d
f = b + c
g = e + f
```

Arithmetic operations:

```
b + c  → 1
b + c  → 2
a * d  → 3
b + c  → 4
e + f  → 5
```

Total = 5 operations

Optimized:

```
t1 = b + c
e = t1 * t1
g = e + t1
```

Arithmetic operations:

```
b + c   → 1
t1*t1   → 2
e + t1  → 3
```

Comparison:

Parameter	**Original**	**Optimized**
Addition	4	2
Multiplication	1	1
Total Arithmetic Operations	5	3
Operations Saved	–	2
Reduction	–	40%

Code:

```
from graphviz import Digraph

dag = Digraph("DAG")

# Leaf nodes
```

```
dag.node("b", "b")
dag.node("c", "c")

# Common expression b + c
dag.node("plus1", "+\nb + c")

dag.edge("b", "plus1")
dag.edge("c", "plus1")

# Variables using the common expression
dag.node("a", "a")
dag.node("d", "d")
dag.node("f", "f")

dag.edge("plus1", "a")
dag.edge("plus1", "d")
dag.edge("plus1", "f")

# Multiplication: e = a * d
dag.node("mul", "*\na * d")
dag.edge("a", "mul")
dag.edge("d", "mul")

dag.node("e", "e")
dag.edge("mul", "e")

# Addition: g = e + f
dag.node("plus2", "+\ne + f")
dag.edge("e", "plus2")
dag.edge("f", "plus2")

dag.node("g", "g")
dag.edge("plus2", "g")
dag
```

Output: